

WEBX Module 1

Compare and contrast Web 1.0, Web 2.0 and Web 3.0

Feature	Web 1.0 (1990s–2000)	Web 2.0 (2004–present)	Web 3.0 (emerging)
Read/Write	Read-only	Read-Write	Read-Write-Execute (own)
Content	Static HTML pages	Dynamic, user-generated	Semantic, 3D, decentralized
Interaction	Information retrieval	Social interaction, comments, likes	Trustless interactions, smart contracts
Data ownership	Websites own data	Platforms own data	User owns data (via blockchain)
Technologies	HTTP, FTP, HTML/CSS	AJAX, JS frameworks, REST, social APIs	Blockchain, IPFS, RDF, AI agents

Example:

```
<!-- Web 1.0: Static -->
<p>Welcome to my page</p>

<!-- Web 2.0: Dynamic + user comments -->
<div id="comments">User: "Great post!"</div>

<!-- Web 3.0: Smart contract interaction -->
<button onclick="wallet.deposit(amount)">Tip with ETH</button>
```

Define and give characteristics of Semantic Web. Explain the components of Semantic Web Stack with diagram.

Semantic Web (by Tim Berners-Lee) is an extension of the current web where information is given well-defined meaning, enabling computers and people to work cooperatively. Data is linked via machine-readable metadata.

Characteristics:

- **Machine-interpretable data** using ontologies (RDF, OWL).
- **Data integration** across heterogeneous sources.
- **Inference & reasoning** (deduce new knowledge).
- **Linked Data** principles (use URIs, HTTP, RDF).

Semantic Web Stack (Layer Cake) – from bottom to top:

Layer	Component	Purpose
1	URI / IRI	Unique identifiers for resources
2	Unicode	Character encoding
3	XML / TURTLE	Syntax for data exchange
4	RDF (Resource Description Framework)	Data model (triples)
5	RDFS (RDF Schema)	Basic class/property hierarchy
6	OWL (Web Ontology Language)	Rich reasoning capabilities
7	SPARQL	Query language for RDF
8	RIF / SWRL	Rules for inference
9	Cryptography / Proof	Trust & verification
10	Trust	User confidence in data

Diagram (text representation):

```

+-----+
|  Trust  |
+-----+
|  Proof  |
+-----+
| Cryptography |
+-----+
|   RIF   |
+-----+
| SPARQL  |
+-----+
|   OWL   |
+-----+
|  RDFS   |
+-----+
|   RDF   |
+-----+
| XML/TURTLE |

```

```
+-----+  
| URI/Unicode |  
+-----+
```

State four significant N-Triples language.

Definition: N-Triples is a line-based, plain-text serialization format for RDF data. Each triple consists of `<subject>` `<predicate>` `<object>` .

Four significant N-Triples examples:

1. Simple literal triple

```
<http://example.org/book/123> <http://purl.org/dc/elements/1.1/title> "RDF  
Primer" .
```

2. Typed literal

```
<http://example.org/people/John> <http://example.org/age>  
"32"^^<http://www.w3.org/2001/XMLSchema#integer> .
```

3. Language-tagged literal

```
<http://example.org/Paris> <http://example.org/label> "Paris"@en .
```

4. Blank node triple

```
_:b1 <http://www.w3.org/1999/02/22-rdf-syntax-ns#type>  
<http://example.org/Person> .
```

Characteristics of N-Triples:

- One triple per line
 - Full URIs in angle brackets
 - Literals in double quotes
 - Ends with . (space + dot)
-

Define Clickstream Analysis and state its applications.

Clickstream analysis is the process of collecting, analyzing, and reporting the sequence of clicks (pages, links, buttons, etc.) made by a user while browsing a website or application. It records the digital path a visitor takes.

Data captured: referrer URL, timestamp, IP, session ID, click coordinates, page exit, bounce.

Applications:

Domain	Application
E-commerce	Identify drop-off points in checkout funnel; optimize product placement
Content sites	Determine most engaging articles; personalize recommendations
UX design	Heatmaps of clicks to redesign confusing layouts
Advertising	Track ad-to-purchase paths (attribution modeling)
Cybersecurity	Detect abnormal click patterns (bots, fraud)
SEO	Understand which landing pages retain users; reduce bounce rate

Example (log entry):

```
2025-05-12T10:15:23Z user_id=456 session=S001 referrer=/home page=/products/laptop
action=click -> add_to_cart
```

Tools: Google Analytics, Adobe Analytics, Matomo, custom event logging.

Explain steps involved in Web Analytics Process.

The Web Analytics Process (following the **WEB** model or standard cycle):

Step	Name	Description
1	Define Goals	KPIs: conversion rate, bounce rate, avg session duration, revenue per visitor
2	Collect Data	Deploy tracking code (JavaScript pixel), server logs, SDKs
3	Process & Enrich	Filter bots, sessionize, map to user IDs, handle cookies/GDPR
4	Store & Organize	Data warehouse (BigQuery, Snowflake) or analytics platform
5	Analyze	Segmentation, funnel analysis, cohort analysis, anomaly detection
6	Report & Visualize	Dashboards (Looker Studio, Tableau), automated alerts
7	Act & Optimize	A/B testing, personalization, UX changes, ad spend reallocation

Code snippet (tracking event):

```
// Step 2 implementation
function trackEvent(category, action, label) {
  if (typeof gtag !== 'undefined') {
    gtag('event', action, { event_category: category, event_label: label });
  } else {
    console.log('[analytics]', category, action, label);
  }
}
trackEvent('Checkout', 'click', 'promo_code_field');
```

Explain different types of Web Analytics.

Type	Focus	Metrics	Tools/Methods
Off-site analytics	Web-wide presence, competition, backlinks	Domain authority, share of voice, brand mentions	SEMrush, Ahrefs, BuzzSumo
On-site analytics	User behavior on your property	Pageviews, session duration, funnel conversion	Google Analytics, Heap, Mixpanel
Real-time analytics	Live user activity	Active users now, live events, streaming clicks	Google Analytics Real-time, Snowplow
Predictive analytics	Forecasting future behavior	Churn probability, LTV prediction, next click	ML models (logistic regression, time series)
Custom event analytics	Specific user interactions (non-pageview)	Button clicks, video plays, form submissions	Segment, RudderStack, custom JS events

Example of custom event analytics code:

```
// Track play button click
document.getElementById('video-play').addEventListener('click', () => {
  analytics.track('Video Played', { title: 'Tutorial', duration_sec: 120 });
});
```

Define RIA and explain different characteristics of RIA.

RIA (Rich Internet Application):

A web application that has many of the characteristics of desktop software, delivering a rich, responsive user experience within a browser, often using client-side frameworks and asynchronous communication.

Characteristics of RIA:

Characteristic	Description
Rich UI	Drag-drop, sliders, animations, modal dialogs, canvases
Asynchronous communication	AJAX/fetch – no full page reload (data in background)
Client-side processing	Logic runs in browser (JavaScript, WebAssembly), reducing server load
Responsive & fast	Immediate feedback (e.g., form validation while typing)
Offline capability	Service workers + IndexedDB (Progressive Web App extension)
Cross-platform	Same code runs on Windows, macOS, Linux, Android, iOS
Stateful	Maintains application state (e.g., shopping cart without page refresh)

Example (RIA characteristic – async form submit):

```
// No page reload
document.querySelector('form').addEventListener('submit', async (e) => {
  e.preventDefault();
  const res = await fetch('/api/submit', { method: 'POST', body: new
FormData(e.target) });
  const data = await res.json();
  document.getElementById('result').innerHTML = `Saved: ${data.id}`;
});
```

Frameworks: React, Angular, Vue.js, Svelte, Blazor, Flutter Web.

Legacy: Flash, Silverlight (now deprecated).